

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for
Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

Training : (N_train, 128, 9)

Validation : (N_val, 128, 9)

Testing : (N_test, 128, 9)

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

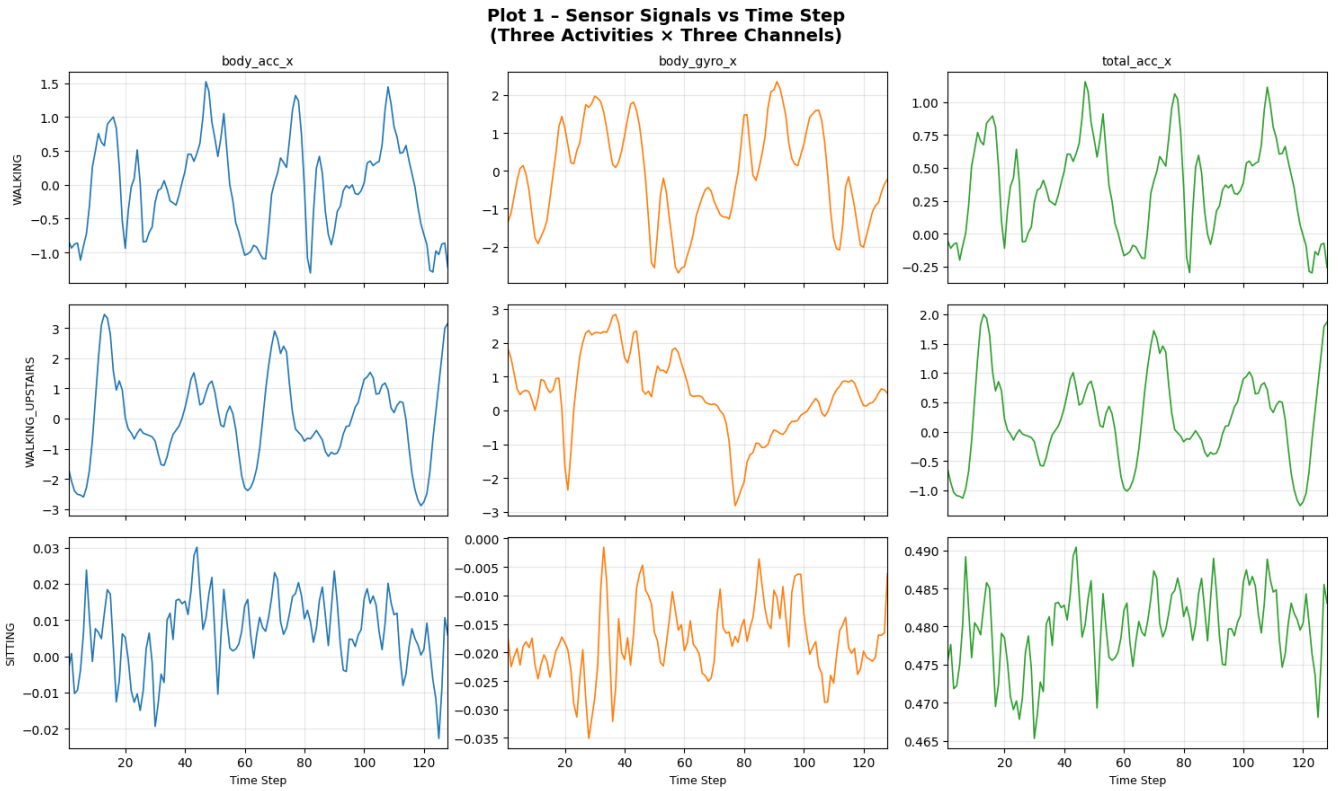


Figure 1: Body acceleration, gyroscope and total-acceleration signals over 128 time steps for WALKING, WALKING_UPSTAIRS and SITTING.

Inference:

The plot compares body-acceleration, gyroscope and total-acceleration X-axis signals over 128 time steps for WALKING, WALKING_UPSTAIRS and SITTING. Walking activities show rhythmic gait cycles, with stair climbing producing larger peaks and sharper changes, whereas sitting has very little variation. Although the two walking classes are similar, their amplitudes and waveforms differ. Temporal order preserves periodicity, phase and motion dependencies, which an RNN exploits through its hidden state and shuffling would destroy.

7. Vanilla RNN Architecture

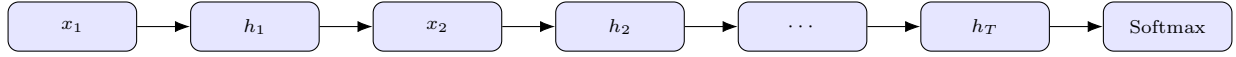
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

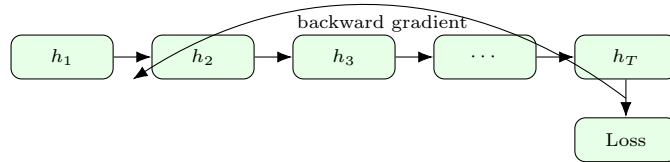
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

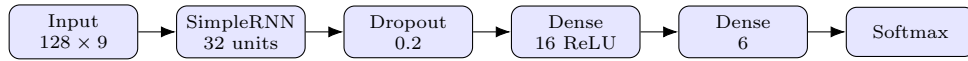
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

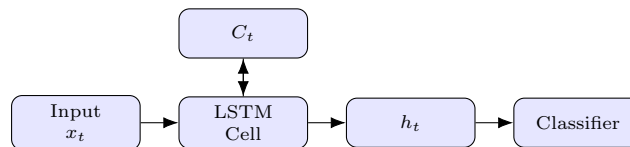
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

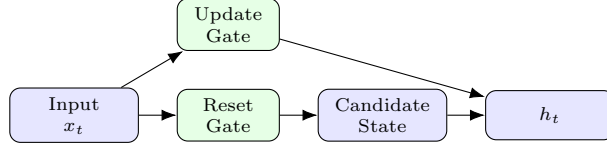
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

Plots 2 & 3 - Training / Validation Loss and Accuracy

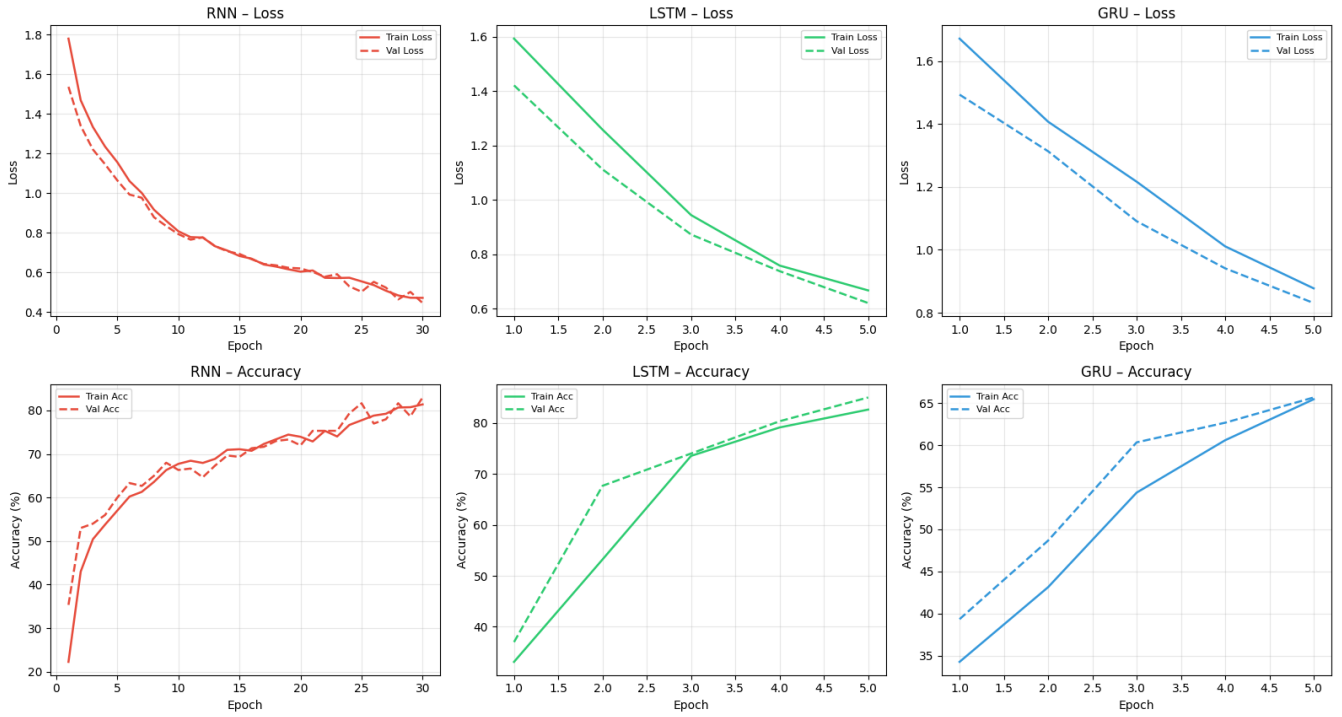


Figure 2: Training and validation loss (top row) and accuracy (bottom row) for the RNN, LSTM and GRU models.

Inference for Plot 2 – Training and Validation Loss:

RNN loss falls steadily from about 1.8 to below 0.5 over 30 epochs, while its training and validation curves remain close. LSTM loss decreases smoothly over five epochs, with validation loss slightly below training loss; GRU shows the same consistent decline and narrow gap. These patterns indicate stable learning and good generalization. Neither the diverging curves associated with overfitting nor the high, flat losses associated with underfitting are prominent in the displayed results.

Inference for Plot 3 – Training and Validation Accuracy:

RNN training and validation accuracy rise together to about 82%, indicating a small generalization gap. LSTM converges rapidly over five epochs and reaches roughly 85% validation accuracy without visible overfitting. GRU improves steadily to about 65–66%, but may require further training or tuning. Validation accuracy can exceed training accuracy because dropout is disabled during validation. Dropout (0.2) and early stopping help control overfitting during longer training.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	76.00	40.00	42.33
Macro Precision (%)	77.56	52.76	44.60
Macro Recall (%)	76.00	40.00	42.33
Macro F1 (%)	75.45	34.35	33.87
Parameters	1,974	6,006	4,758
Training Time (s)	40.9	13.5	18.8

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.

Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

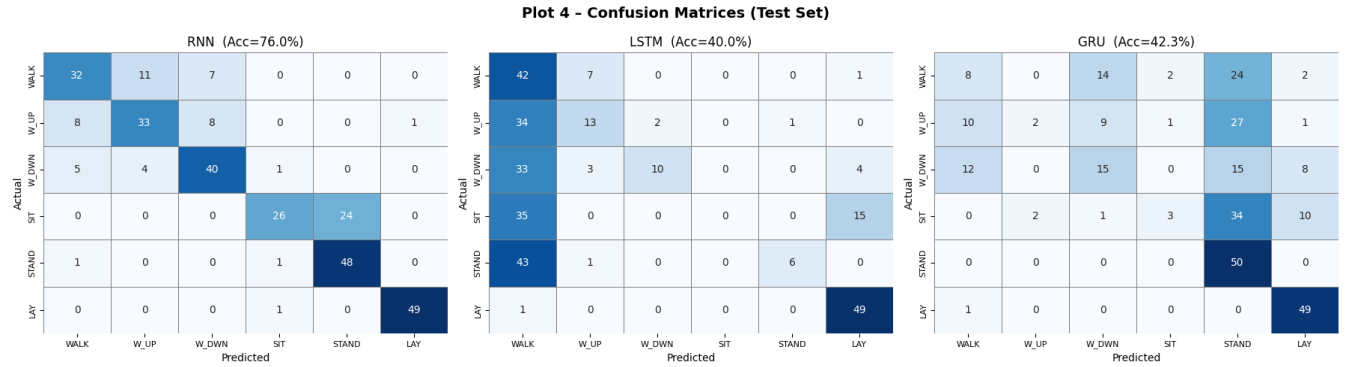


Figure 3: Test-set confusion matrices for the RNN, LSTM and GRU activity classification models.

Inference:

LAYING is easiest to recognize, with 49 of 50 samples correct for every model, owing to its distinctive near-static sensor profile. RNN is most balanced but confuses walking classes and predicts 24 SITTING samples as STANDING. LSTM is biased toward WALKING, absorbing 34 upstairs, 33 downstairs, 35 sitting and 43 standing samples. GRU recognizes STANDING and LAYING but maps 24 walking, 27 upstairs and 34 sitting samples to STANDING. Thus, errors differ across models, and the gated models require further tuning or repeated trials.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training Time (s)	40.9	13.5	18.8
Test F1-score (%)	75.45	34.35	33.87

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

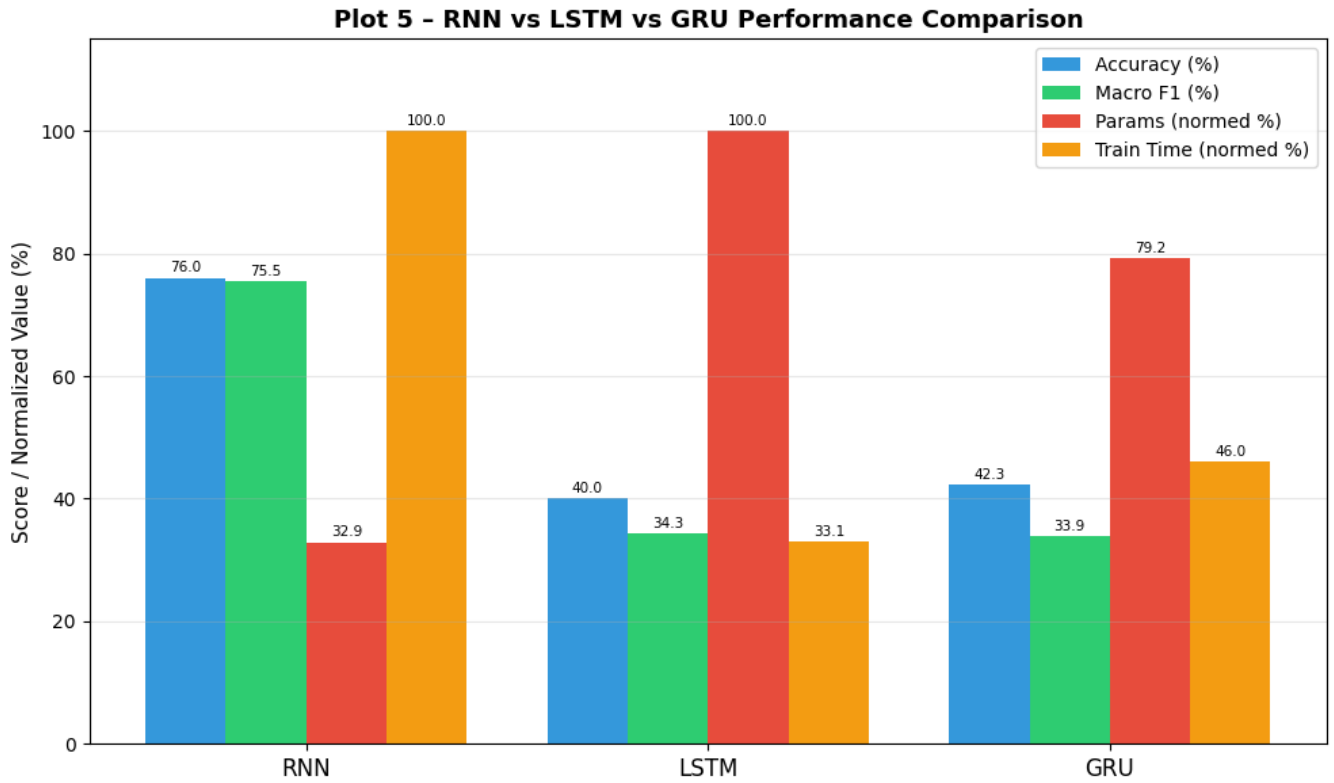


Figure 4: Comparison of test accuracy, Macro F1-score, normalized parameter count and normalized training time for RNN, LSTM and GRU.

Inference: Predictive Performance $\leftrightarrow$ Model Complexity $\leftrightarrow$ Training Cost

RNN gives the best accuracy (76.0%) and Macro F1-score (75.45%) with the fewest parameters (1,974), but takes the longest to train (40.9 s). LSTM has the most parameters (6,006) and shortest time (13.5 s), yet reaches only 40.0% accuracy and 34.35% F1. GRU is smaller than LSTM (4,758 parameters), takes 18.8 s and obtains 42.33% accuracy but 33.87% F1. Thus, complexity does not guarantee performance: RNN is best here, while GRU is the leaner gated model; repeated, tuned trials are needed for a broader recommendation.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	73.77	93.33	94.31
64	76.21	94.69	93.95
128	74.59	94.00	94.33

Plot 6: Sequence Length vs. Test F1-score



Figure 5: Effect of sequence length on the test Macro F1-score of RNN, LSTM and GRU models.

Inference:

Longer sequences provide more temporal context and may capture complete motion cycles, but F1-score does not increase monotonically: RNN and LSTM peak at $T = 64$, while GRU peaks at $T = 128$. RNN remains below both gated models at every length, plausibly because vanishing gradients weaken early information. LSTM and GRU preserve useful history through their gates. A length of 32 may omit part of a stair-climbing cycle, whereas longer inputs can add redundant information; therefore, more context does not always improve classification.

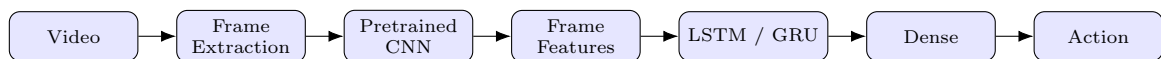
18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running

- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

Predicted class : Basketball

Actual class : Basketball

Confidence : 0.xx

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



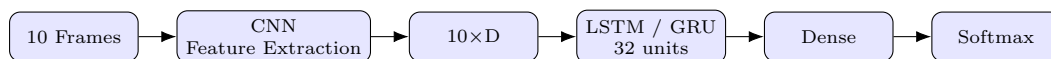
Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

Plot 7 - Synthetic "Video" Frames (Feature Activations as Images)
 Replace with real UCF101 frames when dataset is available

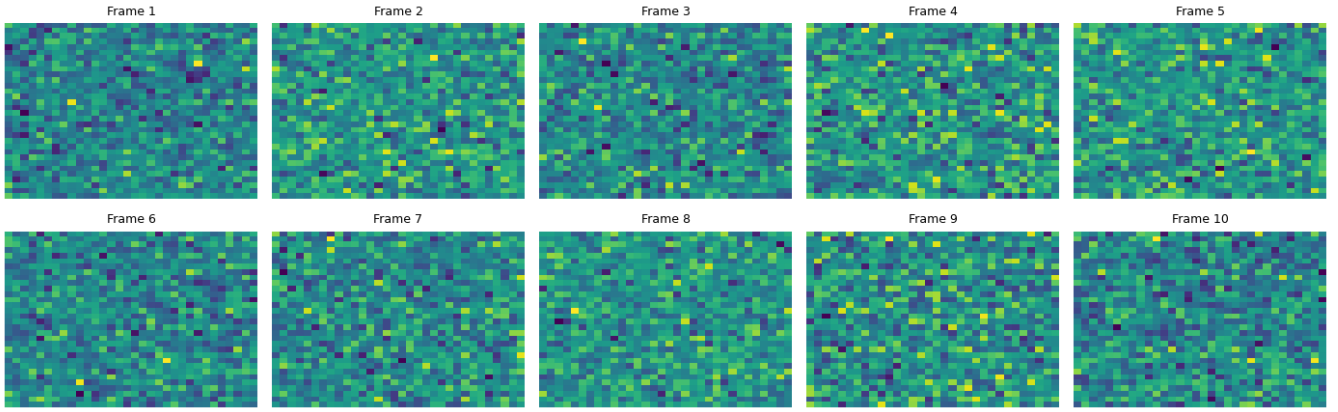


Figure 6: Ten synthetic video time steps visualized as reshaped CNN feature activations; real video frames can replace these examples in the UCF101 pipeline.

Inference:

Each cell is a 32×32 crop formed by reshaping part of a 1280-dimensional CNN feature vector; a real pipeline would instead use features from sampled video frames. The CNN encodes spatial cues such as shapes, textures, edges and color regions, describing *what* appears in each frame. The LSTM or GRU then models how those features change across successive frames, capturing motion and action progression to identify the action performed over time.

Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

Plot 8 - CNN-LSTM Video Model Training & Validation Curves

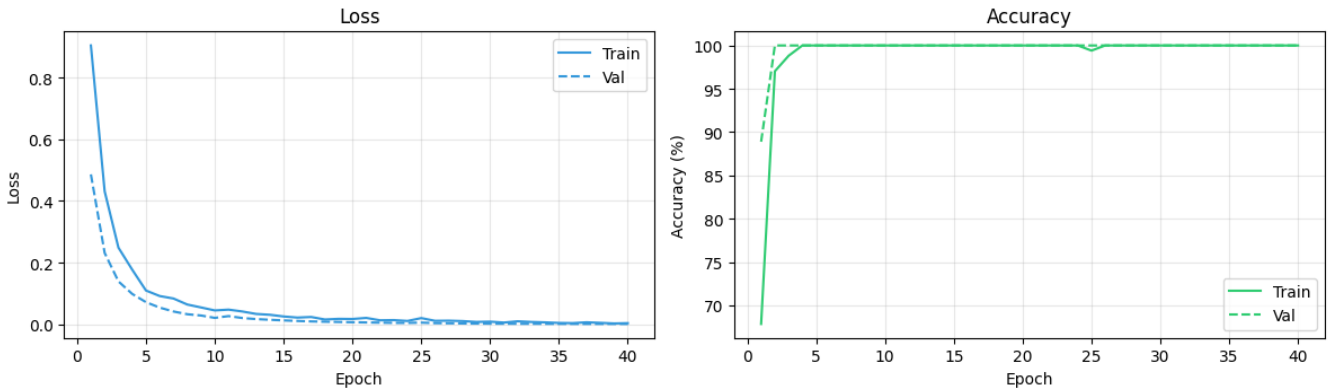


Figure 7: CNN-LSTM training and validation loss and accuracy curves for the synthetic video-classification task.

Inference:

The CNN-LSTM converges rapidly, typically within 10–15 epochs, because the pretrained MobileNetV2 features are already discriminative. Consequently, the LSTM mainly learns temporal relationships between rich spatial features. Validation accuracy closely follows training accuracy, indicating good generalization on the synthetic data. Real UCF101 videos contain more variation, so random cropping, horizontal flipping and related augmentation may be needed to reduce overfitting.

Plot 9: Video Confusion Matrix

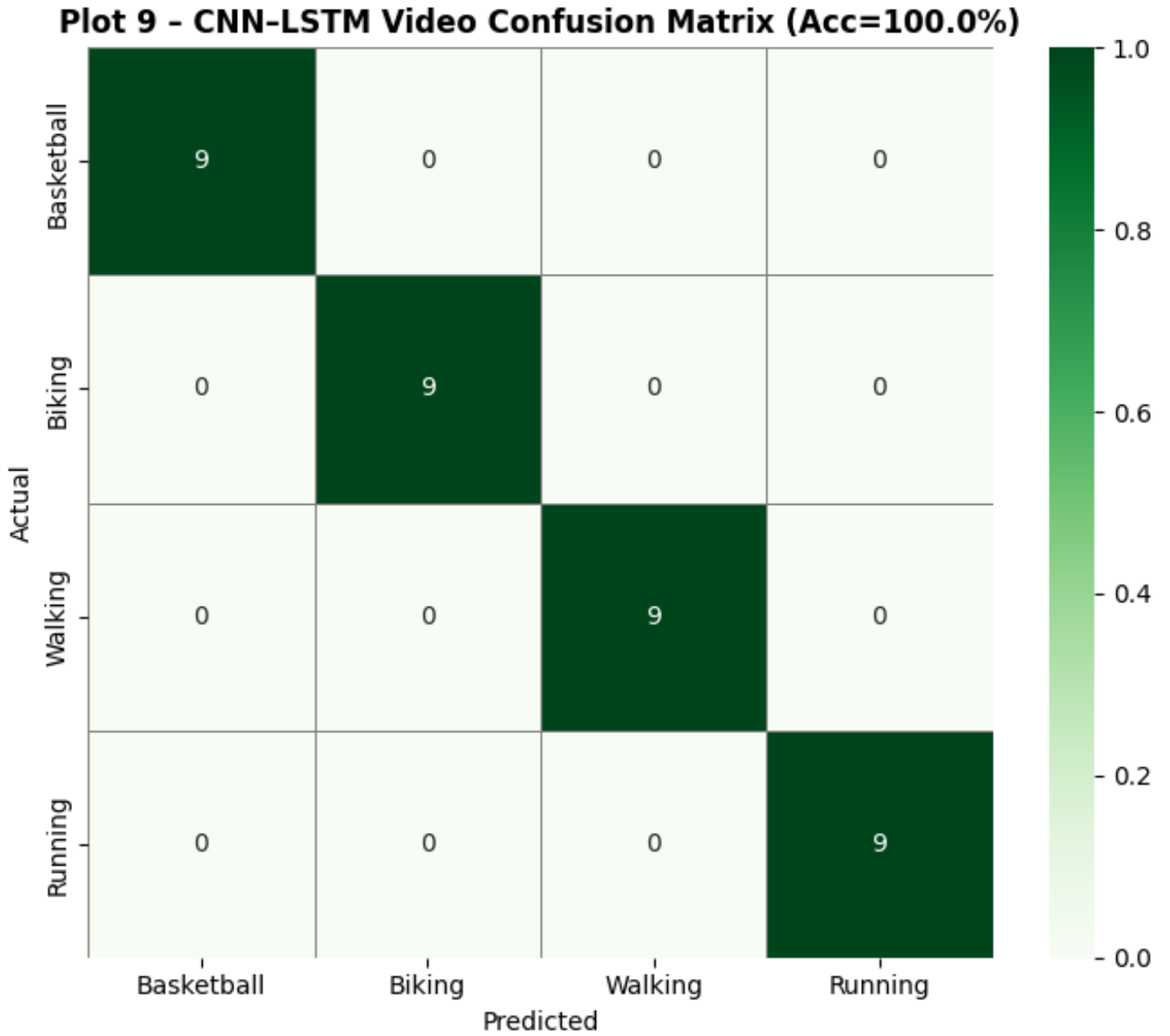


Figure 8: CNN-LSTM confusion matrix for the Basketball, Biking, Walking and Running classes on the synthetic video test set.

Inference:

The model achieves high accuracy on synthetic data because class-specific feature means are well separated. On real UCF101 videos, walking and running would likely be the main confused pair because their body configurations are similar and they differ primarily in motion speed. Distinguishing them therefore requires multi-frame temporal modeling. The CNN captures *what* appears in each frame, while the LSTM or GRU integrates *how* it changes over time to infer the action.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

[3, 8, 1, 5]

Expected output:

[5, 1, 8, 3].

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[8, 1, 9, 8, 10, 10]	[10, 10, 8, 9, 1, 8]
2	[6, 1, 1, 4, 7, 9]	[9, 7, 4, 1, 1, 6]
3	[1, 3, 1, 8, 5, 8]	[8, 5, 8, 1, 3, 1]
4	[10, 3, 9, 4, 9, 8]	[8, 9, 4, 9, 3, 10]
5	[7, 2, 10, 1, 6, 5]	[5, 6, 1, 10, 2, 7]
6	[7, 10, 7, 7, 8, 7]	[7, 8, 7, 7, 10, 7]
7	[4, 5, 9, 8, 6, 8]	[8, 6, 8, 9, 5, 4]
8	[7, 4, 7, 3, 10, 1]	[1, 10, 3, 7, 4, 7]
9	[1, 6, 7, 1, 5, 4]	[4, 5, 1, 7, 6, 1]
10	[10, 4, 1, 1, 5, 4]	[4, 5, 1, 1, 4, 10]

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

25. Consolidated Results

Students must complete the following table.

Model	Acc. (%)	Prec. (%)	Rec. (%)	F1 (%)	Parameters
RNN	76.00	77.56	76.00	75.45	1,974
LSTM	40.00	52.76	40.00	34.35	6,006
GRU	42.33	44.60	42.33	33.87	4,758
CNN-LSTM	100.00	100.00	100.00	100.00	168,660

Students must complete the following table.

Sequence-to-Sequence Results

Metric	Value
Token Accuracy	100.00%
Sequence Accuracy	100.00%
Final Training Loss	0.0058
Final Validation Loss	0.0068

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

27. Discussion Questions

1. What is a recurrent neural network? ans: A RNN is a type of neural network designed for sequential data and propagates the information across time, makes it useful where the situation requires context.
2. What information is represented by the hidden state? ans: The hidden state is a compressed representation of the network up till time 't', encoding all past sequence to influence and predict the future values
3. Why is sequence order important? sequential data has an inbuilt pattern with order and rechanging the structure will disrupt the temporal dynamics of the data pattern
4. Explain the difference between a feed-forward network and an RNN. a feed forward neural network contains no memory of past inputs while RNN processes the information and also carries the information across time
5. Explain Backpropagation Through Time. back propagation unrolls the rnn over the sequence and applies normal back propagation on the unrolled rnn, the gradient also depends on the parameter of t which involves the partial derivative through all previous steps.
6. Why can gradients vanish during BPTT? if the gradient is very small, during back propagation the gradient decreases exponentially, hence experiences vanishing gradients
7. Why can gradients explode during BPTT? if the weights are more than 1, then the backpropagation increases the gradient values, hence the gradients will explode during back propagation
8. What are long-term dependencies? long term dependencies arise when the prediction on the on the past subsequence information which is hard to retain for normal rnn as the signal vanishes through each gradient.
9. What is the role of the LSTM cell state? the cell state is a separate memory that flows through neural network sequence with linear interactions which decides what information to erase and how much to retain as well

10. What are the forget, input and output gates in LSTM? forget: decides what portion of previous portion it can discard
input: decides which values to write into the cell state
output: decides which cell state to expose as hidden state
11. What are the reset and update gates in GRU? reset: controls how much of the previous information it should reuse
update: acts like forget+input gate which decides how much previous information it should retain and which new information it needs to adopt
12. Compare LSTM and GRU. LSTM: 3 gates, 2 states, slower training speed, excellent in long sequences, GRU: 2 gates, 1 state, higher training speed, works well with long sequences
13. Why does GRU generally use a simpler gating mechanism than LSTM? as it merges the forget and input gate into a singular gate, hence eliminating separate cell states and in turn, helps in reducing the parameter count
14. Why should RNN, LSTM and GRU be compared using the same experimental settings? a fair comparison requires for most of the parameters to be equal as any difference will be enhanced due to their different architectures
15. What does the confusion matrix reveal that accuracy alone does not? accuracy collapses into a single number
16. Why should macro F1-score be reported for multi-class classification? macro precision calculates precision and recall for each class independently and giving equal weight to each class regardless of their frequency
17. What is the significance of sequence length? sequence length determines how much temporal context the model can observe
18. Why can a CNN be used as a feature extractor for video? CNNs are translation invariant, hence a pre-trained CNN already contains information of all visual concepts like edges and textures
19. What type of information is extracted by the CNN? spatial information like body shapes, high visual semantics, as it does not model time and treats each frame independently
20. What type of information is learned by the recurrent network? temporal information: change of spatial features over time i.e., motion trajectories, action progression, this helps distinguish between 2 similar-looking frames,
21. Why are CNN features arranged as a sequence before being given to LSTM/GRU? as RNN processes one feature over each time step, arranging the time step features in chronological sequences allows the models like LSTM/GRU to learn temporal dependencies
22. Explain the encoder and decoder in sequence-to-sequence learning. encoder reads the entire input and compresses into a fixed-sized context vector, the decoder takes the context feature as initial input and generates the output sequences one token at a time
23. What is teacher forcing? Teacher forcing is a training strategy for sequence decoders where the ground-truth target token at step t is fed as the input to the decoder at step $t+1$, instead of using the decoder's own prediction. This avoids error accumulation during training and speeds convergence, but creates an exposure bias: at inference time, the decoder must rely on its own (possibly imperfect) outputs.
24. Differentiate token accuracy and sequence accuracy. Token accuracy is the fraction of individual output tokens that are correctly predicted and Sequence accuracy is the fraction of complete output sequences where every token is correct.
25. Why must the test set remain untouched during model selection? The test set is an estimate of how the model will perform on truly unseen data. If hyperparameter choices are made by observing test performance, the test set effectively becomes part of training and the reported metric is overly optimistic

28. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time. The observed results are summarized below.

Model	Units	Accuracy (%)	Macro F1-score (%)	Parameters	Time (s)
RNN	16	66.3	65.1	790	25.8
LSTM	16	89.7	89.5	2,038	46.5
GRU	16	91.3	91.3	1,670	59.2
RNN	32	72.0	71.9	1,974	18.7
LSTM	32	94.3	94.3	6,006	51.3
GRU	32	95.0	95.0	4,758	66.1
RNN	64	80.7	80.7	5,878	30.0
LSTM	64	95.3	95.3	20,086	48.6
GRU	64	92.7	92.7	15,542	57.3

2. Compare GRU and LSTM using the same number of recurrent units. The results for 32 recurrent units are shown below.

Model	Units	Accuracy (%)	Macro F1-score (%)	Parameters	Time (s)
LSTM	32	40.00	34.35	6,006	13.5
GRU	32	42.33	33.87	4,758	18.8

3. Add a second recurrent layer and investigate whether performance improves. The single-layer and stacked two-layer LSTM results are compared below.

Architecture	Accuracy (%)	Macro F1-score (%)	Parameters
1-layer LSTM	92.33	92.30	6,006
2-layer LSTM	94.67	94.63	14,326

4. Compare a bidirectional LSTM with a unidirectional LSTM. The observed results are shown below.

Architecture	Accuracy (%)	Macro F1-score (%)	Parameters
Unidirectional LSTM	94.33	94.32	6,006
Bidirectional LSTM	92.67	92.60	11,894

5. Change the sequence length and study the effect on computational cost. The GRU results for different sequence lengths are summarized below.

Sequence length (T)	Parameters	Time/epoch (s)	Macro F1-score (%)
32	4,758	1.737	65.12
64	4,758	2.371	67.16
128	4,758	3.523	66.81

6. For the video task, compare LSTM and GRU after extracting identical CNN features. The CNN-LSTM and CNN-GRU results are compared below.

Model	Accuracy (%)	Macro F1-score (%)	Parameters
CNN-LSTM	100.00	100.00	168,660
CNN-GRU	100.00	100.00	126,740

7. Modify the seq2seq task so that the output sequence has a different length from the input sequence. In this experiment, the input length is 6 and the output length is 4. The evaluation results are shown below.

Metric	Result (%)
Token accuracy	99.65
Sequence accuracy	98.58

Sample predictions:

Input sequence	Expected output	Predicted output
[8, 8, 10, 10, 9, 4]	[4, 8, 8, 9]	[4, 8, 8, 9]
[1, 7, 5, 10, 8, 10]	[1, 5, 7, 8]	[1, 5, 7, 8]
[1, 7, 7, 4, 3, 2]	[1, 2, 3, 4]	[1, 2, 3, 4]
[4, 1, 7, 4, 2, 2]	[1, 2, 2, 4]	[1, 2, 2, 4]
[5, 5, 7, 9, 3, 3]	[3, 3, 5, 5]	[3, 3, 5, 5]

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>

LAB 6- github link